Configuration and backends

Siddharth M. Bhatia · siddharth@sidmb.com
August 8, 2026

Contents

Configuration layers 1
Default command 1
Target setup 2
Manage saved targets 2
SSH 3
RunPod 3
Lambda Cloud 3
Slurm 4
Project setup and synchronization 4

Configuration layers

User configuration lives at `~/.fwd/config.toml`. A project `.fwd/config.toml` deep-merges over it; target-specific settings have the highest precedence.

Use generated output instead of copying a static exhaustive example:

```
fwd config                          # effective values and their
     source files
fwd config --example               # complete commented TOML
fwd config --example runpod        # one backend
fwd config --schema                # JSON Schema for tools and editors
fwd config set sync.delete false
fwd config set --project forwarding.ports 3000 8080:3000
fwd config rm --project forwarding.ports
```

Default command

Claude is the built-in default for a newly created session. Change it by scope:

```
fwd default codex
fwd default --project claude
fwd default --target runpod -- python -m agent
```

Precedence is target, project, user, then built-in Claude. An explicit agent or command on `fwd up` overrides the default for that launch.

Target setup

Humans can use the interactive wizard:

```
fwd targets add
fwd targets add runpod
```

`fwd setup` is a permanent alias for `fwd targets add` and accepts exactly the same arguments.

Scripts and coding agents should pass all required values. Missing values produce the exact flags needed instead of opening a prompt:

```
fwd targets add ssh --host my-box --target-name work
fwd targets add runpod --data-center-id US-GA-1 --target-name pod
fwd targets add lambda --instance-type gpu_1x_a10 --ssh-key-name my-
     public-key --target-name lambda-gpu
fwd targets add slurm --login-host login.example.edu --user me --remote-
     base /scratch/me/fwd
```

Manage saved targets

`fwd targets` operates on the `[targets.NAME]` tables in configuration; `fwd ls` and `fwd rm` operate on the sessions running against them.

```
fwd targets ls                  # name, backend, key connection detail,
     default marker
fwd targets ls pod              # only targets whose name contains "pod"
fwd targets info work           # resolved values, which are defaults,
     and tracked sessions
fwd targets update work         # re-run setup prefilled with the
     current values
fwd targets update work --host new-box   # non-interactive single-field
     edit
fwd targets rm work             # remove the config entry only
```

`update` and `rm` open a picker when no target is named, and fail with the available names when one is misspelled or when nothing can answer a prompt. `rm` requires `--force` non-interactively, mirroring `fwd config rm`. Removing the default target repoints `default_target` at the sole remaining target, or clears it when the choice would be a guess. Removal never touches remote compute or session state: destroy those with `fwd rm`.

Inspect currently valid provider machine strings with `fwd up --machines` or `fwd up TARGET --machines`. A one-launch `--machine/-m` value must exactly match an available provider value.

SSH

SSH targets connect to machines you already control. Prefer an OpenSSH host alias so `~/.ssh/config` owns the user, port, identity, and proxy settings:

```
fwd up --target my-box
fwd targets add ssh --host my-box --target-name work
```

RunPod

RunPod requires `runpodctl` 2.6.0 or newer and an authenticated account. CPU is the compute default. Persistent Secure Cloud storage is enabled by default for CPU and GPU sessions; `fwd stop` terminates the disposable Pod while retaining the per-session network volume, and `fwd rm` deletes both. `persistent = false` opts into disposable storage.

Persistent targets require a datacenter. GPU targets also require an exact GPU choice and an appropriate image:

```
fwd targets add runpod --target-name pod --data-center-id US-GA-1
fwd up pod --machines
```

Lambda Cloud

Lambda uses its HTTPS API directly and requires a locally available `LAMBDA_API_KEY`, instance type, and registered SSH key. Interactive setup

can securely store a pasted key or a reference to a key file. The API key is never copied to the remote instance.

Persistent filesystems are enabled by default and survive instance termination. `fwd stop` terminates compute while retaining storage; `fwd rm` deletes both. Lambda cannot perform remote stop-after because that would require copying the broad account API key to the instance.

```
export LAMBDA_API_KEY='...'
fwd targets add lambda --target-name lambda-gpu --instance-type gpu_1x_a10
     --ssh-key-name my-public-key
fwd doctor --target lambda-gpu
```

Slurm

Slurm targets connect through a login node and run the allocation inside persistent tmux. Use shared scratch storage for the project, tools, and caches; do not use a quota-constrained home directory.

```
fwd targets add slurm --target-name hpc --login-host login.hpc.example.edu
     --user me --remote-base /scratch/me/fwd
```

Allocation, partition, account, proxy jump, and environment-module settings are cluster-specific. On Slurm, ordinary `fwd send` commands run on the login node; invoke `srun` when work belongs inside an allocation.

Project setup and synchronization

fwd detects Python, JavaScript, and Swift Package Manager projects, reuses compatible remote tools, and installs missing requirements in user-controlled persistent locations. Commit an idempotent `.fwd/setup.sh` for project-specific setup that built-in toolchains do not cover.

Common synchronization settings:

```
[sync]
exclude = [".venv", "node_modules", "dist"]
use_gitignore = true
```

```
delete = true
max_size_gb = 1.0
continuous = false          # Mutagen-backed continuous sync; per target:
     [targets.NAME] continuous_sync

[forwarding]
ports = ["3000", "8080:3000"]
```

Setting `sync.exclude` replaces the default list rather than extending it. See `fwd config --example` for every field and current default.

`sync.continuous` is the default for continuous synchronization. Any target may override it with `continuous_sync`, which is deliberately three-state: omit it to inherit `sync.continuous`, or set `true`/`false` to decide for that target regardless of the global default.

```
[sync]
continuous = true            # most targets sync continuously

[targets.hpc]
backend = "slurm"
continuous_sync = false      # ...except this one
```

`fwd sync on` and `fwd sync off` write exactly this key for you, so it rarely needs editing by hand. Continuous sync never includes `.git`; use `fwd push` and `fwd pull` for repository state.

© 2026 Siddharth M. Bhatia. All rights reserved.